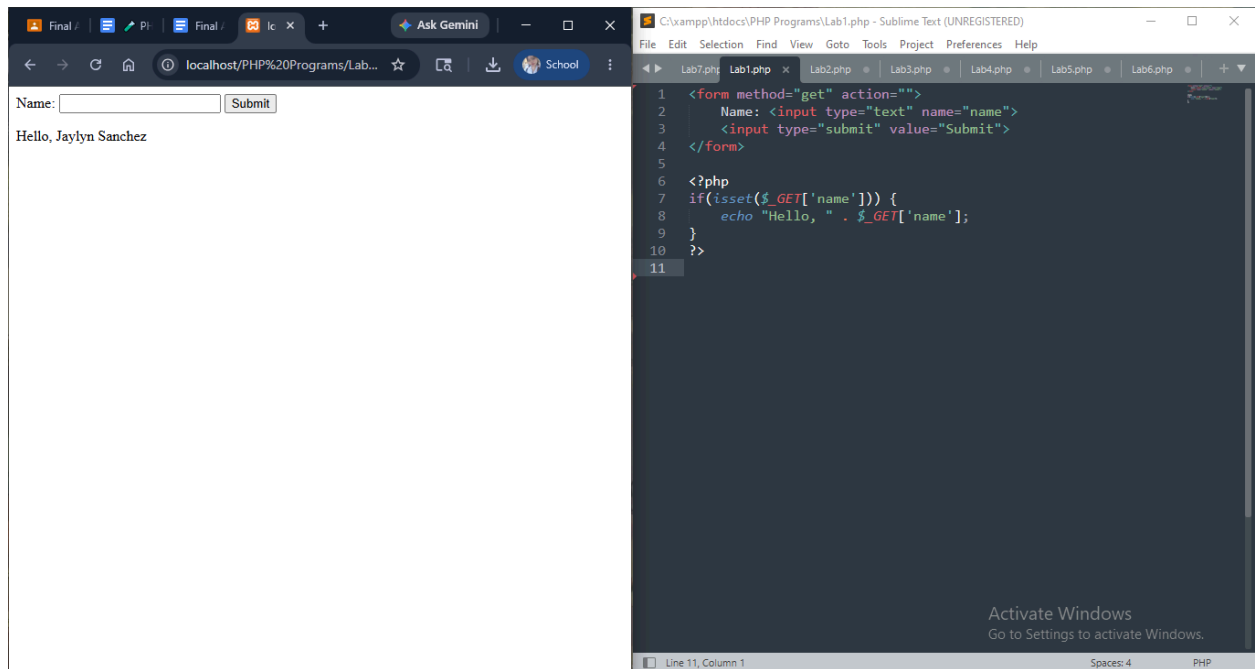
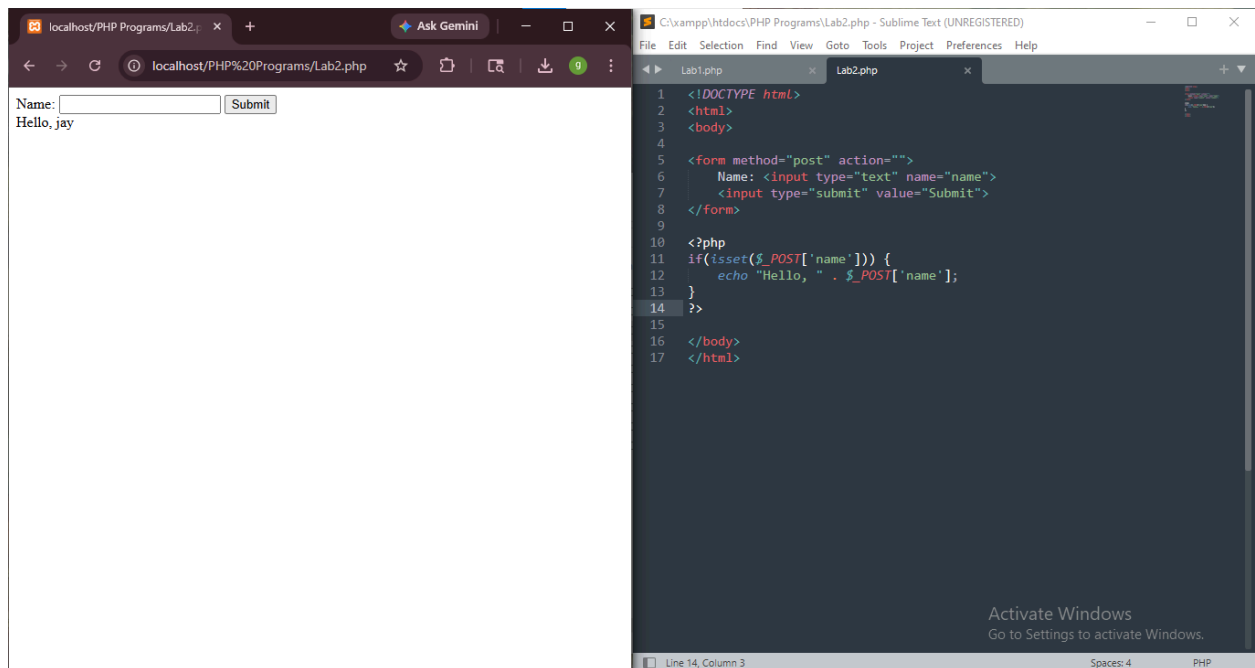


Lab 1: Basic Form Creation



Lab 2: Using POST Method



Lab 3: Multiple Inputs

The screenshot displays a web browser on the left and a code editor on the right. The browser shows a form with two text inputs labeled 'Name:' and 'Email:', a 'Submit' button, and the output: 'Name: Jaylyn Sanchez' and 'Email: jfsanchez@paterostechnologicalcollege.edu.ph'. The code editor shows the PHP code for Lab3.php, which uses `<input type="text">` for the inputs and `<input type="submit" value="Submit">` for the button. The PHP code checks for the presence of 'name' and 'email' in `$_POST` and echoes their values.

```
<form method="post" action="">
  Name: <input type="text" name="name"><br><br>
  Email: <input type="text" name="email"><br><br>
  <input type="submit" value="Submit">
</form>

<?php
if(isset($_POST['name']) && isset($_POST['email'])) {
    echo "Name: " . $_POST['name'] . "<br>";
    echo "Email: " . $_POST['email'];
}
?>
```

Lab 4: Form Validation (Required Fields)

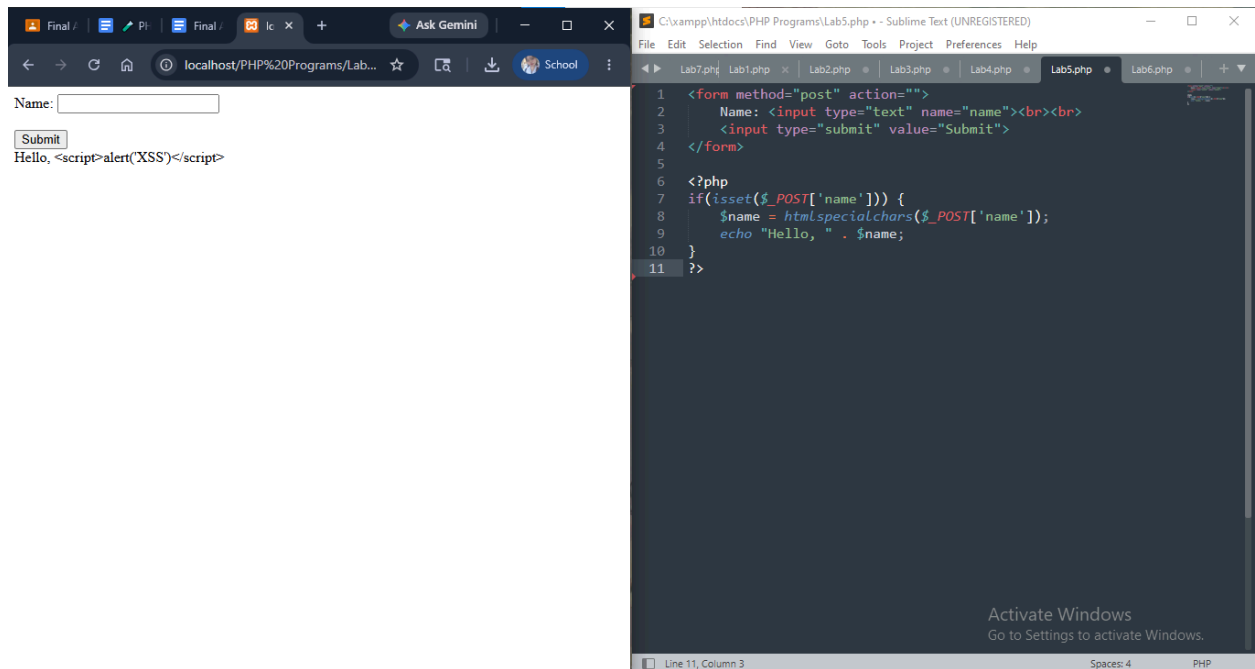
The screenshot displays a web browser on the left and a code editor on the right. The browser shows the same form as Lab 3, but the output is 'Name: Jaylyn Sanchez' and 'Email is required'. The code editor shows the PHP code for Lab4.php, which uses `<input type="text">` for the inputs and `<input type="submit" value="Submit">` for the button. The PHP code checks if the request method is 'POST' and then validates if 'name' or 'email' are empty. If empty, it echoes an error message; otherwise, it echoes the value.

```
<form method="post" action="">
  Name: <input type="text" name="name"><br><br>
  Email: <input type="text" name="email"><br><br>
  <input type="submit" value="Submit">
</form>

<?php
if($_SERVER["REQUEST_METHOD"] == "POST") {
    if(empty($_POST["name"])) {
        echo "Name is required<br>";
    } else {
        echo "Name: " . $_POST["name"] . "<br>";
    }

    if(empty($_POST["email"])) {
        echo "Email is required<br>";
    } else {
        echo "Email: " . $_POST["email"];
    }
}
?>
```

Lab 5: Prevent XSS

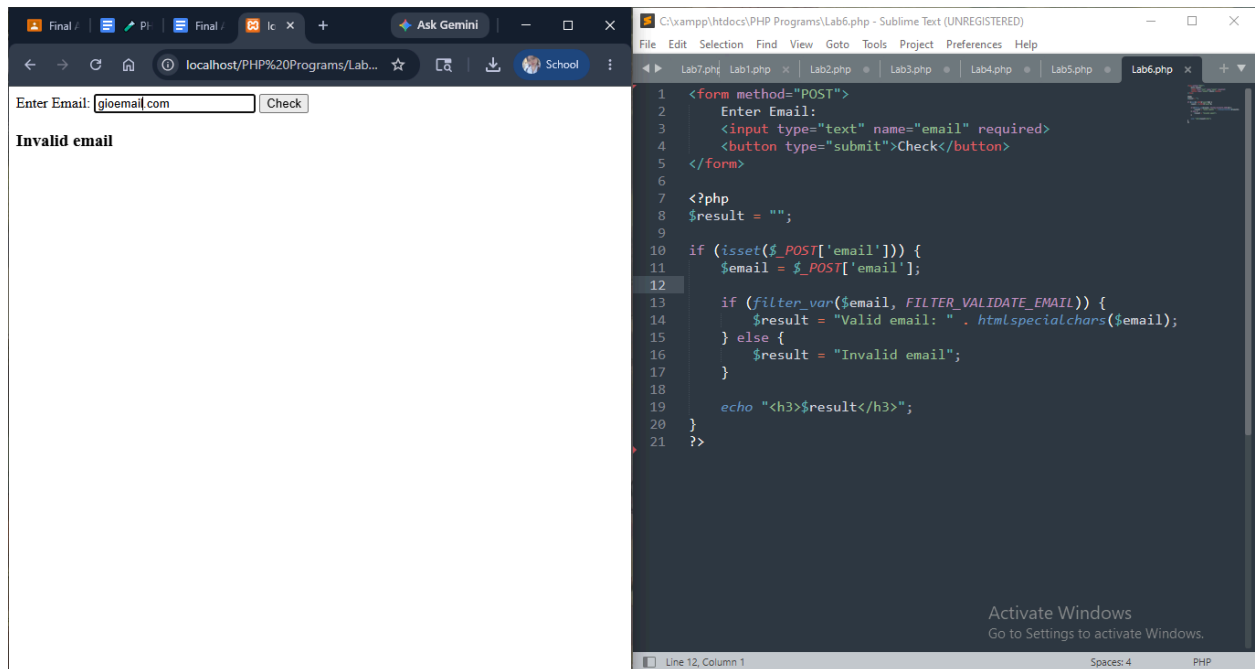


The browser window shows a web application running on localhost. It has a form with a label "Name:" and a text input field. Below the input field is a "Submit" button. After clicking the button, the page displays "Hello, <script>alert('XSS')</script>".

The Sublime Text editor shows the code for Lab5.php:

```
1 <form method="post" action="">
2   Name: <input type="text" name="name"><br><br>
3   <input type="submit" value="Submit">
4 </form>
5
6 <?php
7 if(isset($_POST['name'])) {
8   $name = htmlspecialchars($_POST['name']);
9   echo "Hello, " . $name;
10 }
11 ?>
```

Lab 6: Email Validation

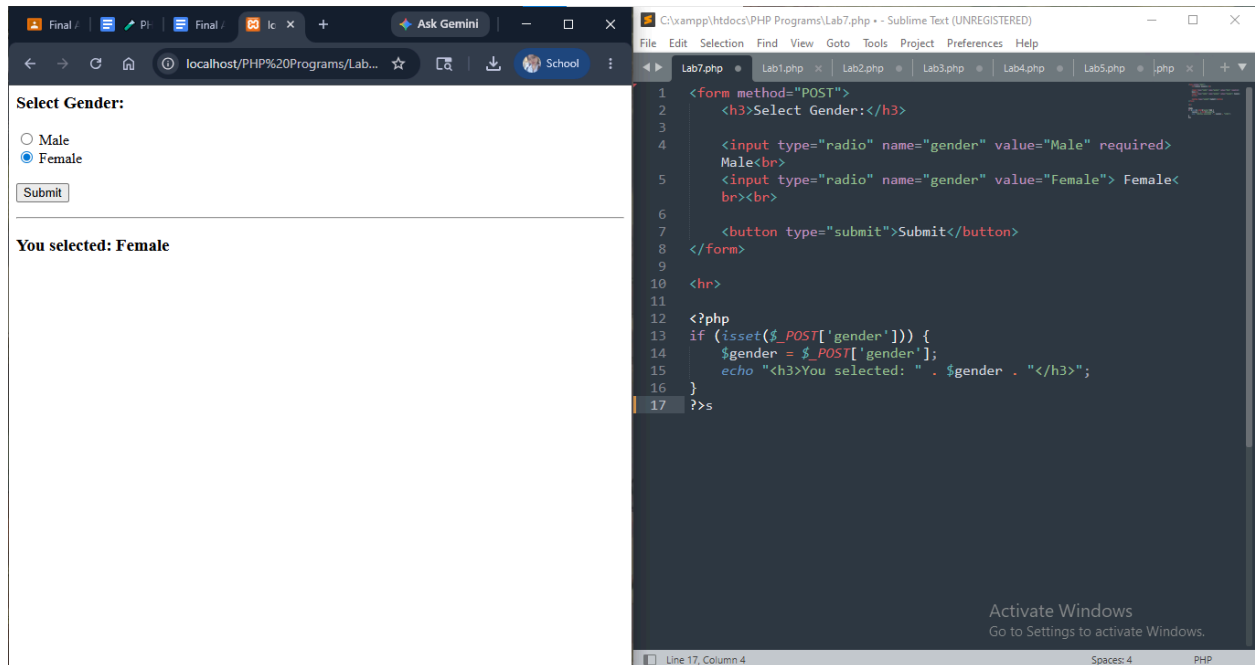


The browser window shows a web application running on localhost. It has a form with a label "Enter Email:" and a text input field containing "gloemail.com". Below the input field is a "Check" button. After clicking the button, the page displays "Invalid email".

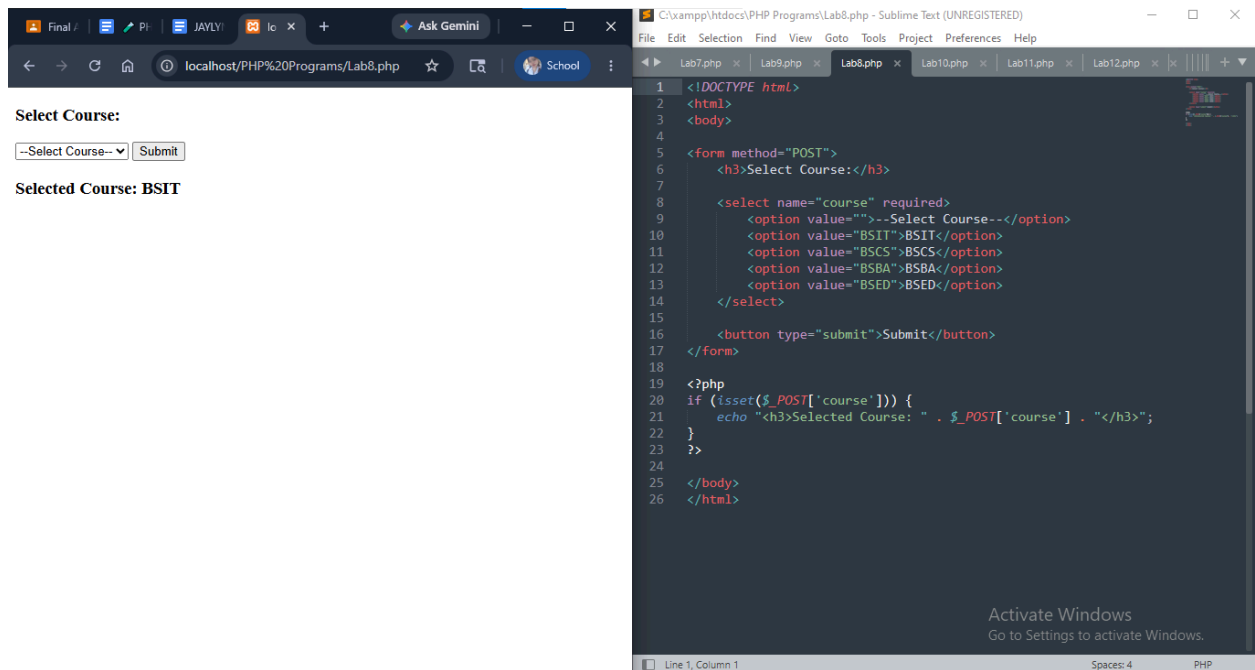
The Sublime Text editor shows the code for Lab6.php:

```
1 <form method="POST">
2   Enter Email:
3   <input type="text" name="email" required>
4   <button type="submit">Check</button>
5 </form>
6
7 <?php
8 $result = "";
9
10 if (isset($_POST['email'])) {
11   $email = $_POST['email'];
12
13   if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
14     $result = "Valid email: " . htmlspecialchars($email);
15   } else {
16     $result = "Invalid email";
17   }
18
19   echo "<h3>$result</h3>";
20 }
21 ?>
```

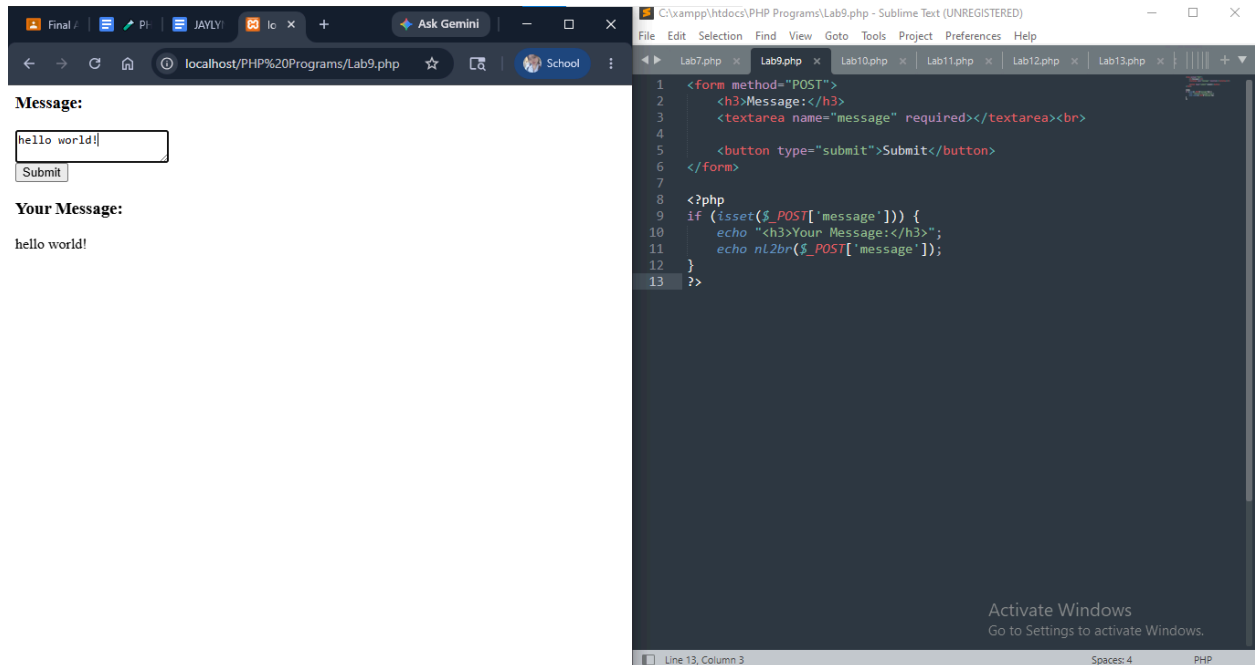
Lab 7: Radio Buttons



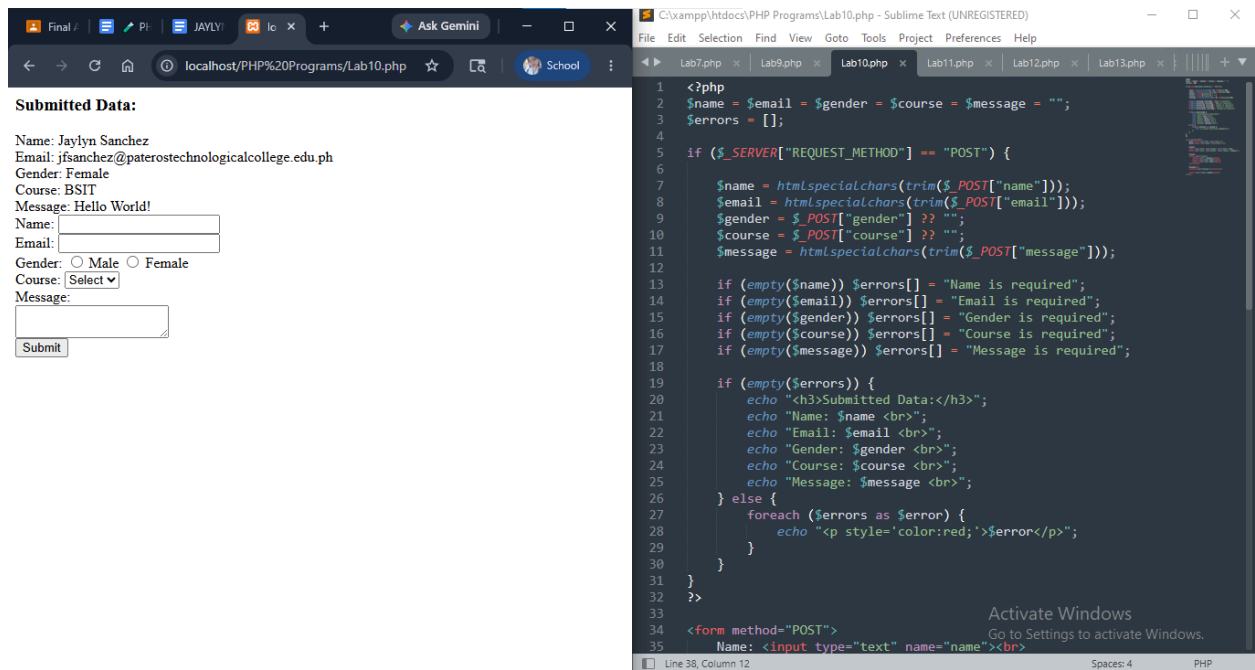
Lab 8: Dropdown Menu



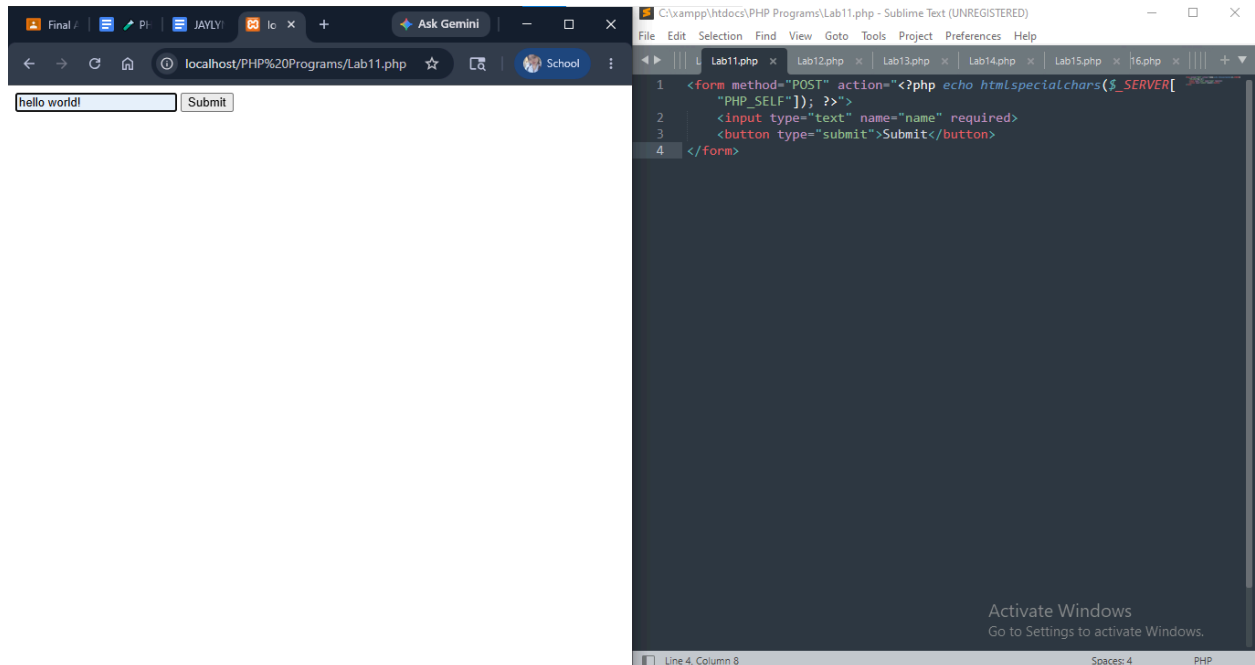
Lab 9: Textarea Input



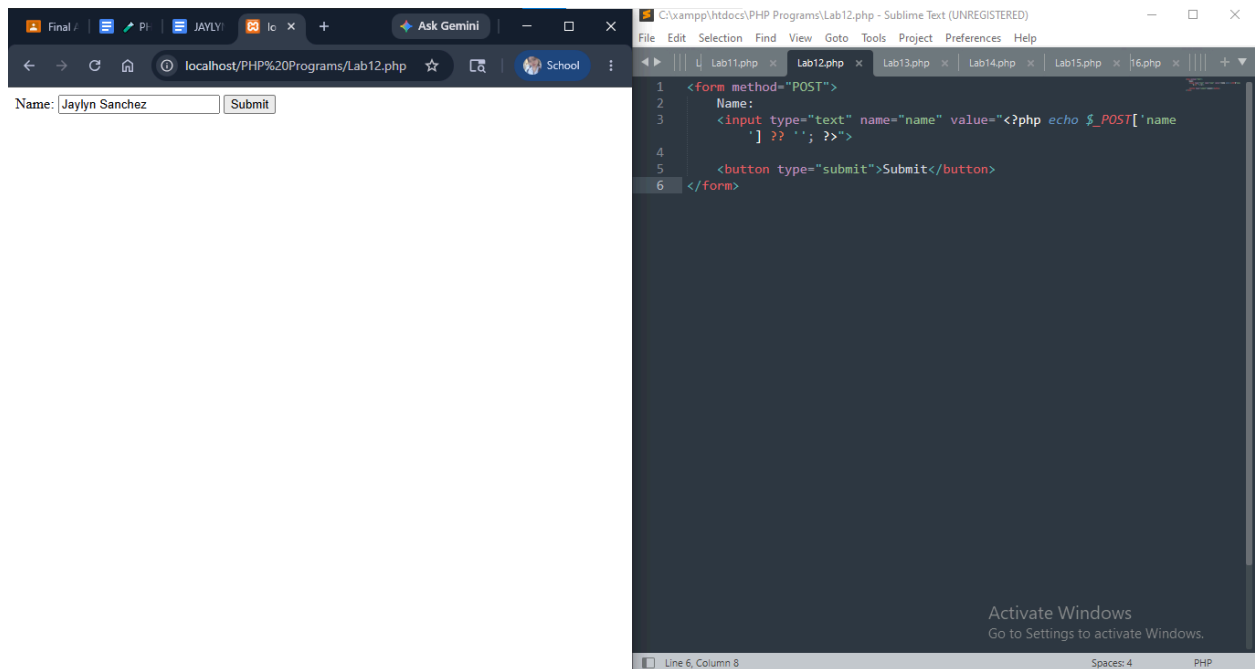
Lab 10: Combined Form



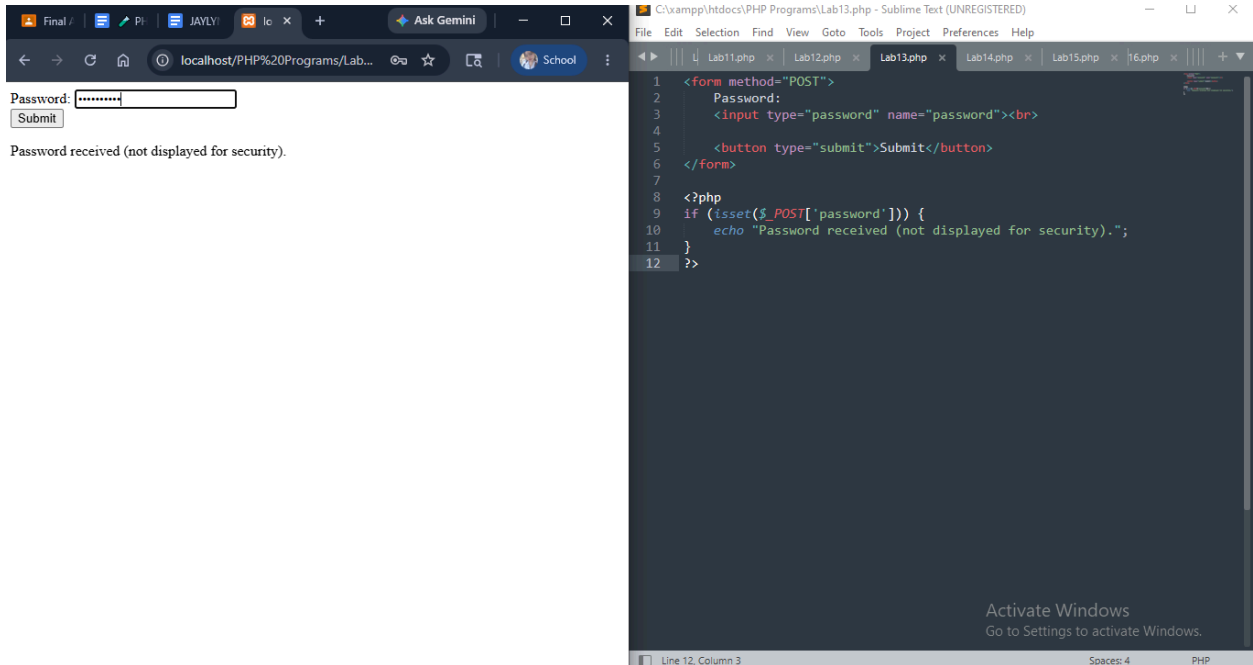
Lab 11: Self-Processing Form: It is safer because it submits the form to the same page while also protecting the website from harmful scripts.



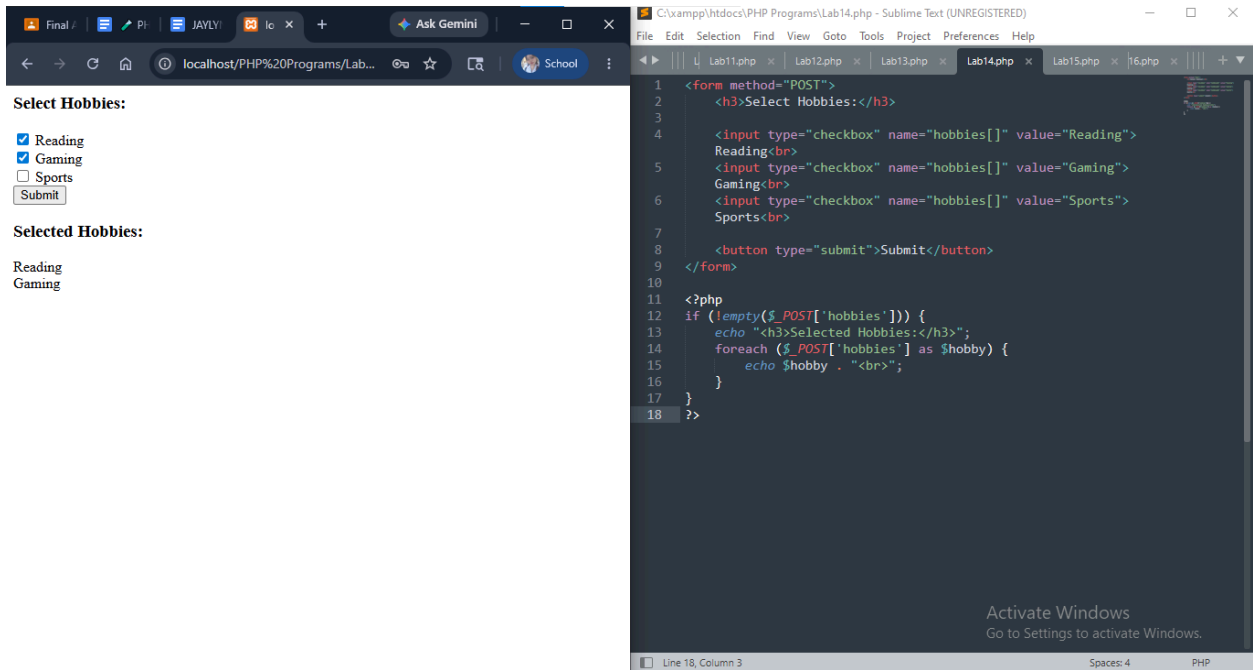
Lab 12: Sticky Form



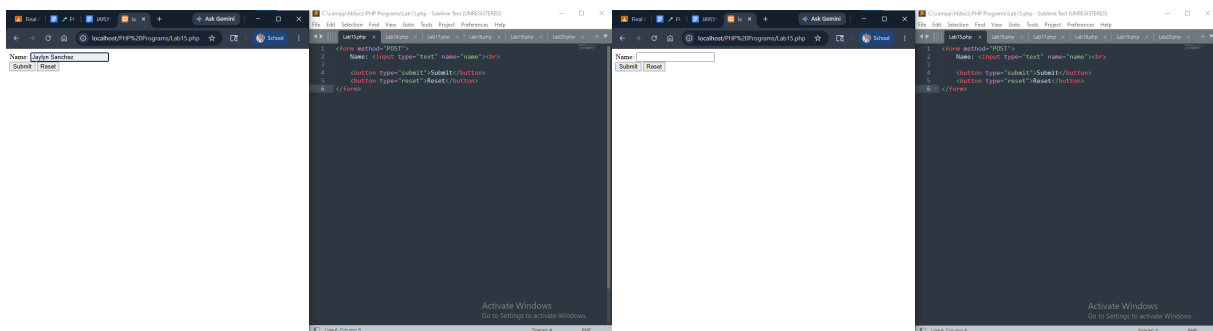
Lab 13: Password Input



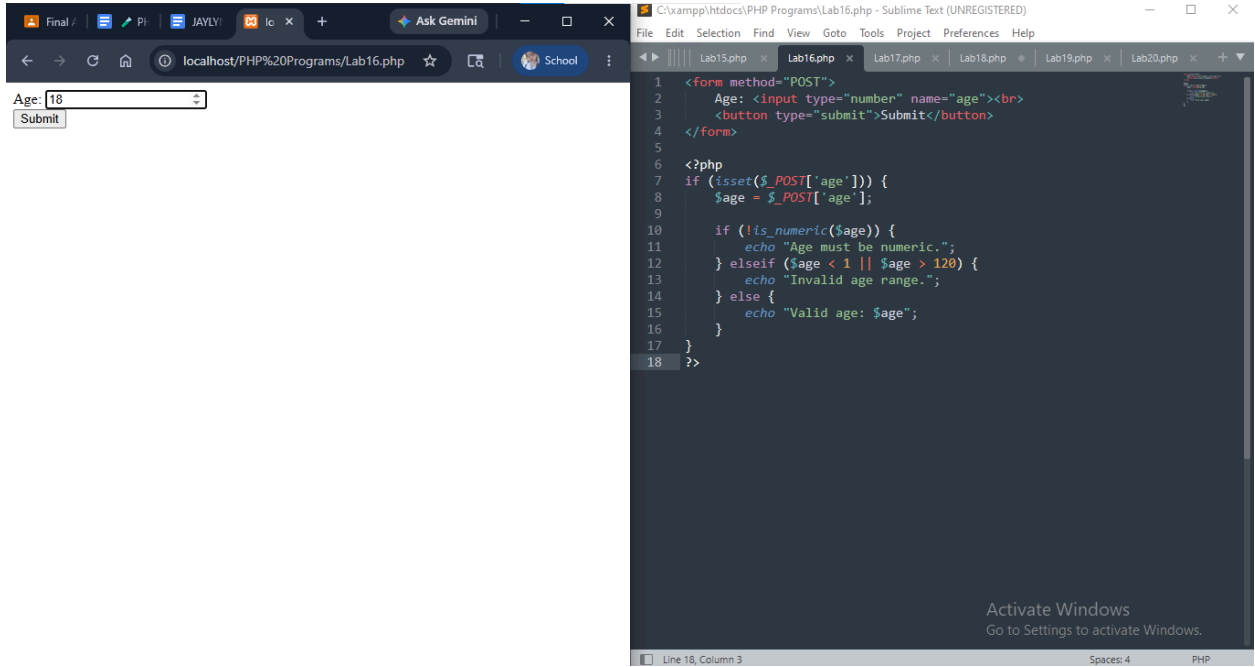
Lab 14: Checkbox Input



Lab 15: Form Reset Button



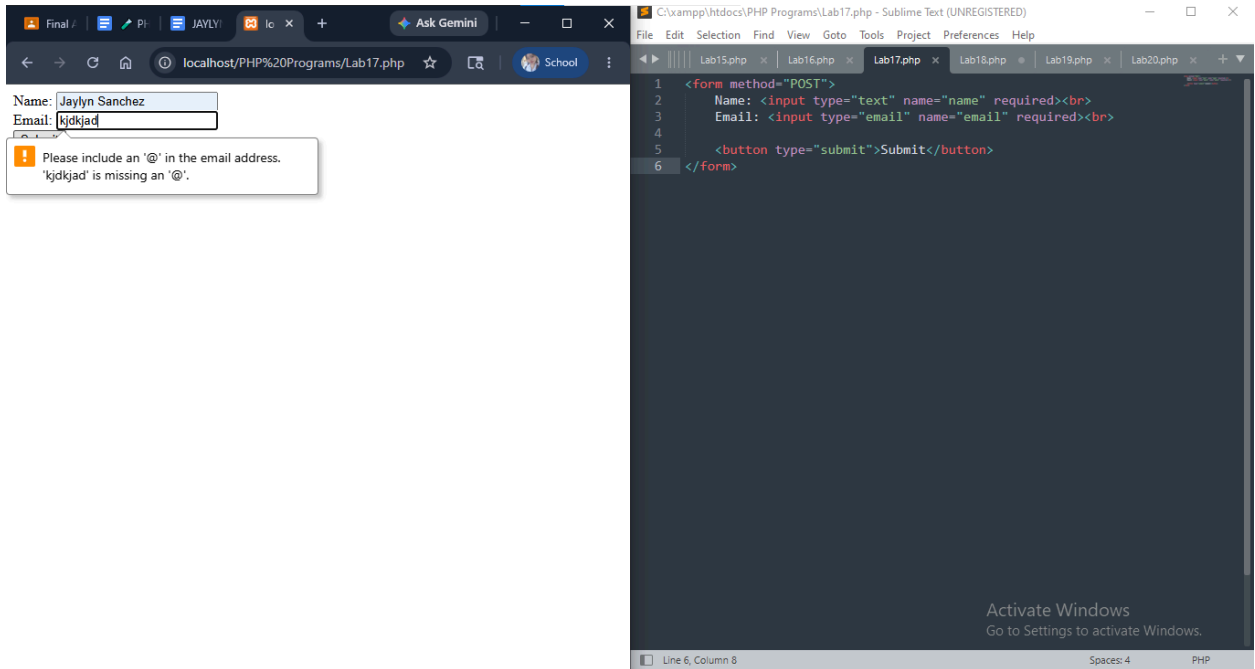
Lab 16: Number Input Validation



The browser window displays a form with the label "Age:" and a text input field containing the value "18". Below the input field is a "Submit" button. The Sublime Text editor shows the PHP code for Lab16.php, which implements a POST method form with an "Age" input and a "Submit" button. The code uses PHP to validate the age input, checking if it is numeric and within the range of 1 to 120. If the input is invalid, it echoes an error message; otherwise, it echoes the valid age.

```
1 <form method="POST">
2   Age: <input type="number" name="age"><br>
3   <button type="submit">Submit</button>
4 </form>
5
6 <?php
7 if (isset($_POST['age'])) {
8     $age = $_POST['age'];
9
10    if (!is_numeric($age)) {
11        echo "Age must be numeric.";
12    } elseif ($age < 1 || $age > 120) {
13        echo "Invalid age range.";
14    } else {
15        echo "Valid age: $age";
16    }
17 }
18 ?>
```

Lab 17: Required Attribute (HTML)



The browser window displays a form with the label "Name:" and a text input field containing the value "Jaylyn Sanchez". Below it is the label "Email:" and a text input field containing the value "kjdkjad". A validation message is displayed below the email field: "Please include an @ in the email address. 'kjdkjad' is missing an '@'." The Sublime Text editor shows the PHP code for Lab17.php, which implements a POST method form with "Name" and "Email" inputs and a "Submit" button. The code uses PHP to validate the email input, checking if it contains an "@" symbol. If the input is invalid, it echoes an error message; otherwise, it echoes the valid email.

```
1 <form method="POST">
2   Name: <input type="text" name="name" required><br>
3   Email: <input type="email" name="email" required><br>
4   <button type="submit">Submit</button>
5 </form>
```

Lab 18: GET vs POST Comparison

The screenshot shows a web browser on the left and a code editor on the right. The browser displays a form with two input fields, each labeled "Name:". The first field has a "Submit GET" button, and the second field has a "Submit POST" button. The code editor shows the PHP code for the form. It includes two form sections: one for GET requests and one for POST requests. The GET form has a text input named "get_name" and a "Submit GET" button. The POST form has a text input named "post_name" and a "Submit POST" button. The PHP code uses `$_GET` and `$_POST` to handle the submitted data and echoes the output.

```
1 <form method="GET">
2     Name: <input type="text" name="get_name">
3     <button type="submit">Submit GET</button>
4 </form>
5
6 <?php
7 if (isset($_GET['get_name'])) {
8     echo "GET Output: " . $_GET['get_name'];
9 }
10 ?>
11
12 <form method="POST">
13     Name: <input type="text" name="post_name">
14     <button type="submit">Submit POST</button>
15 </form>
16
17 <?php
18 if (isset($_POST['post_name'])) {
19     echo "POST Output: " . $_POST['post_name'];
20 }
21 ?>
22
```

```
C:\xampp\htdocs\PHP Programs\Lab18.php - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

Lab15.php x Lab16.php x Lab17.php x Lab18.php e Lab19.php x Lab20.php x + -

1 <form method="GET">
2     Name: <input type="text" name="get_name">
3     <button type="submit">Submit GET</button>
4 </form>
5
6 <?php
7 if (isset($_GET['get_name'])) {
8     echo "GET Output: " . $_GET['get_name'];
9 }
10 ?>
11
12 <form method="POST">
13     Name: <input type="text" name="post_name">
14     <button type="submit">Submit POST</button>
15 </form>
16
17 <?php
18 if (isset($_POST['post_name'])) {
19     echo "POST Output: " . $_POST['post_name'];
20 }
21 ?>
22
```

Lab 19: Form with File Structure

The image is a composite of three screenshots illustrating a web development task. The top-left screenshot shows a web browser at the URL `localhost/PHP%20Programs/pro...` displaying the output of a PHP script: "Your name is: Jaylyn Sanchez". The bottom-left screenshot shows the same browser at `localhost/PHP Programs/form...` displaying a simple HTML form with a text input field labeled "Name:" and a "Submit" button. The right-side screenshot shows two code editors. The top editor, Sublime Text, displays the PHP code for `process.php`, which checks if the 'name' POST variable is set and echoes its value. The bottom editor, also Sublime Text, displays the HTML code for `form.php`, which uses the `<form>` tag to create a POST form that submits to `process.php`, including an input field for the name and a submit button.

C:\xampp\htdocs\PHP Programs\process.php - Sublime Text (UNREGISTERED)

```
1 <?php
2 if (isset($_POST['name'])) {
3     echo "Your name is: " . $_POST['name'];
4 }
5 ?>
```

C:\xampp\htdocs\PHP Programs\form.php - Sublime Text (UNREGISTERED)

```
1 <form method="POST" action="process.php">
2     Name: <input type="text" name="name">
3     <button type="submit">Submit</button>
4 </form>
```

Line 4, Column 8 Spaces: 4 PHP

Activate Windows
Go to Settings to activate Windows.

Line 5, Column 3 Spaces: 4 PHP

Lab 20: Mini Project – Student Registration Form

